

Metrics Reference Guide

A plain-English guide to every metric in the log sheets

TIME

RETRIEVAL

FAITHFULNESS

RELEVANCE

BERTSCORE

ACCURACY

Version 2.0 · March 2026

Overview — All Metrics

This document explains every metric recorded by the College Chatbot evaluation system. Each metric card shows **what it measures**, the **exact formula**, **how the calculation works**, and **how to interpret the score**.

Metric	Sheet	Range	One-line summary	Weight in Accuracy
Time Taken (s)	Production	0 – ∞ s	Pure server processing time	—
Latency (s)	Evaluation	0 – ∞ s	Total client round-trip time	—
Server Time (s)	Evaluation	0 – ∞ s	Server time echoed in API response	—
Retrieval Conf. %	Both	0 – 100 %	Semantic match: query vs top chunk	—
Faithfulness %	Evaluation	0 – 100 %	Are claims grounded in the context?	30 %
Relevance %	Evaluation	0 – 100 %	Does the answer address the question?	25 %
Completeness %	Evaluation	0 – 100 %	All parts of the question answered?	20 %
BERTScore F1 %	Evaluation	0 – 100 %	Semantic overlap: answer vs context	15 %
Link Validity	Evaluation	X/Y Valid	Are hyperlinks in the answer live?	—
Accuracy %	Evaluation	0 – 100 %	Weighted final quality score	Final
Source	Both	Text label	Which system answered: RAG / SQL / KB	10 % (routing)

SECTION 1

Timing Metrics

How long did the system take to respond?

Time Taken (s)

**PRODUCTION
SHEET**

WHAT IT MEASURES

The number of seconds the server spent processing one request — from the moment the backend receives the HTTP call to the moment it finishes generating the answer. Network transfer time is NOT included.

FORMULA

$$\text{Time Taken} = T_{\text{end}} - T_{\text{start}}$$

T_{start} = instant /query endpoint receives the request (server clock)

T_{end} = instant answer is fully generated (server clock)

HOW IT WORKS

A Python `time.time()` timer is started at the top of the /query handler. It stops the moment the model returns its final answer — whether that is a SQL result or LLM-generated text. The elapsed seconds are saved to the Production sheet with 4 decimal place precision.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
0 – 1 s	Excellent — SQL / Knowledge Base instant response
1 – 4 s	Good — RAG with fast GPU / cached model
4 – 10 s	Acceptable — normal LLM inference on CPU
> 10 s	Slow — model may be overloaded or cold-starting

■ Time Taken does NOT include network latency or client rendering time.

Latency (s)

EVALUATION
SHEET

WHAT IT MEASURES

The total wall-clock time from when the test script sends the HTTP request to when the final response chunk lands on the client. This includes server processing time PLUS all network overhead.

FORMULA

$$\text{Latency} = T_{\text{received}} - T_{\text{sent}}$$

Both timestamps measured on the CLIENT (prompt_test.py)

For streaming: T_received = arrival of the SSE 'done' event

For SQL/JSON: T_received = full HTTP body received

HOW IT WORKS

In prompt_test.py, time.time() is captured before requests.post() is called. For streaming responses the clock stops when the 'done' SSE event is received. The gap naturally includes all network round-trip time, which on localhost is typically < 10 ms.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
≈ Server Time	Network overhead negligible — local / fast LAN
+ 50–200 ms	Small network gap — normal on local machine
+ 0.5 – 2 s	Noticeable delay — check network or proxy
> Server + 2 s	Large gap — network bottleneck or buffering issue

Server Time (s)

EVALUATION SHEET

WHAT IT MEASURES

The server-side processing time as reported by the backend and sent back inside the API response. This is identical in meaning to Time Taken (Production) but is captured inside the Evaluation sheet so both timing perspectives sit side-by-side for comparison.

FORMULA

$\text{Server Time} = T_{\text{end}} - T_{\text{start}}$ (measured on server)

Streaming: sent inside the SSE 'done' event → { time_taken: X.XXXX }

SQL/JSON: sent in the JSON response field → { time_taken: X.XXXX }

HOW IT WORKS

The backend computes start/end time the same way as Time Taken. It serialises the result and includes time_taken in the response payload. prompt_test.py reads this field and stores it as Server Time — giving you the authoritative server number without any client-side noise.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
≈ Latency	Network is fast — values will be nearly identical
Much < Latency	Network delay is significant; focus on Server Time
Much > Latency	This should not happen — check for clock skew
N/A	Response did not include time_taken field

SECTION 2

Retrieval Quality Metrics

Did the system find the right information?

Retrieval Confidence (%)

BOTH SHEETS

WHAT IT MEASURES

How confident the RAG retriever was when selecting the document chunks to answer a query. A high score means the retrieved content is semantically very close to the question. A low score means the retriever was uncertain and may have returned loosely related material.

FORMULA

$A \cdot B$

$\cos(\theta) = \frac{A \cdot B}{|A| \times |B|}$

$A \cdot B = \sum (a_i \times b_i) \leftarrow \text{dot product (multiply pairs, then sum)}$

$|A| = \sqrt{(a_1^2 + a_2^2 + \dots + a_n^2)} \leftarrow \text{vector length}$

$\theta = 0^\circ \rightarrow \cos = 1.0 \rightarrow \text{identical meaning (score = 100\%)}$

$\theta = 90^\circ \rightarrow \cos = 0.0 \rightarrow \text{completely unrelated (score = 0\%)}$

$\text{Confidence \%} = \cos(\theta) \times 100$

HOW IT WORKS

Both the user's query and every stored document chunk are converted to dense 384-dimension vectors using the all-MiniLM-L6-v2 sentence-transformer model. Cosine similarity measures the angle between the query vector and each chunk vector — zero angle means identical meaning, 90° means unrelated. The similarity of the single best-matching chunk is taken, multiplied by 100, and stored as Retrieval Confidence.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
80 – 100 %	Excellent — retriever found exactly the right content
60 – 79 %	Good — likely relevant, minor gaps possible
40 – 59 %	Moderate — answer may contain hallucinated details
< 40 %	Poor — retriever is guessing; treat answer with caution

■ SQL and Knowledge-Base answers show N/A — no vector retrieval is performed for structured data queries.

Source

BOTH SHEETS

WHAT IT MEASURES

A text label identifying which internal system produced the answer. This tells you the exact data pipeline the chatbot used.

FORMULA

RAG → Vector retrieval from PDF/HTML knowledge base + LLM generation

SQL → Structured query on the student/placement database

KB → Hard-coded Knowledge Base (known college facts)

Hybrid → Combination of SQL result + RAG context

No formula – routing is rule-based + intent classification

HOW IT WORKS

The QueryRouter analyses the intent of every incoming question using keyword and embedding-based classification. Count / list questions go to SQL. Factual, procedural, and policy questions go to RAG or KB. The routing decision is attached to every log row.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
RAG	LLM-generated answer from retrieved document chunks
SQL	Direct database lookup — high precision for numbers
KB	Hardcoded facts — fastest and most reliable
Unknown	Router could not classify the query — review logs

SECTION 3

LLM-as-Judge Metrics

Scored by Gemma 1B using the question, context, and answer

The three metrics below are evaluated by a small local language model (Gemma 1B via Ollama). The judge receives the original question, the retrieved context (up to 600 chars), and the chatbot's answer (up to 600 chars). It returns a JSON object with three 0.0–1.0 scores. These are multiplied by 100 and saved as percentages.

Faithfulness % (LLM)

EVALUATION SHEET

WHAT IT MEASURES

Whether every factual claim in the chatbot's answer is directly supported by the retrieved context. An answer that invents facts not present in the source material will score low — even if those facts happen to be correct in the real world.

FORMULA

Judge prompt: 'Is every claim in the answer supported by the context?'

Judge returns: { "faithfulness": <0.0 - 1.0> }

Faithfulness % = judge_score × 100

No explicit math — Gemma 1B reads context + answer and outputs a strict 0.0–1.0 opinion on whether every claim is grounded.

Weight in Accuracy: 30 % (highest — hallucination is biggest risk)

HOW IT WORKS

The Gemma judge reads the retrieved context and the bot answer side by side. For each factual statement in the answer it checks: can this be traced to the context? Filler phrases like 'please contact the office' are ignored. Only verifiable claims are assessed. A score of 1.0 means every single claim has clear contextual support.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
90 – 100 %	Every statement grounded in source — highly reliable
70 – 89 %	Mostly faithful, minor unsupported details present
50 – 69 %	Some hallucination — review the answer carefully
< 50 %	Significant hallucination — retrieval may be wrong

Relevance % (LLM)

EVALUATION SHEET

WHAT IT MEASURES

How directly the chatbot's answer addresses the actual question asked. A high-relevance answer stays on topic. A low-relevance answer may be truthful but discusses something other than what the user asked.

FORMULA

Judge prompt: 'Does the answer directly address the question?'

Judge returns: { "relevance": <0.0 - 1.0> }

$\text{Relevance \%} = \text{judge_score} \times 100$

No explicit math — Gemma 1B reads question + answer and rates how directly the response addresses what was asked.

Weight in Accuracy: 25 %

HOW IT WORKS

The judge is shown the question and answer (no context). It evaluates: did the response stay on topic and specifically answer what was asked? For example — if a student asks 'What is the library timing?' and the bot responds about fee structure, that is a near-zero relevance answer regardless of how accurate the fee information is.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
90 – 100 %	Laser-focused — exactly what the user asked for
70 – 89 %	Mostly on-topic with minor tangents
50 – 69 %	Partly relevant — answer drifts into unrelated info
< 50 %	Bot answered the wrong question entirely

Completeness % (LLM)

EVALUATION
SHEET

WHAT IT MEASURES

Whether the answer covers ALL aspects of the question, not just part of it. A complete answer addresses every sub-part of a multi-faceted question. Partial answers score lower even if the covered portion is perfectly accurate and relevant.

FORMULA

Judge prompt: 'Does the answer cover all aspects of the question?'

Judge returns: { "completeness": <0.0 - 1.0> }

Completeness % = judge_score × 100

No explicit math – Gemma 1B maps the scope of the question against the scope of the answer and rates coverage 0.0 – 1.0.

Weight in Accuracy: 20 %

HOW IT WORKS

The judge maps the scope of the question against the scope of the answer. Example: 'Tell me about placement statistics' expects total placed, company names, package ranges, and department breakdown. An answer mentioning only the total count scores around 0.3. All four aspects covered scores near 1.0.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
90 – 100 %	Comprehensive — all parts of the question answered
70 – 89 %	Mostly complete — one or two minor aspects missed
50 – 69 %	Partial answer — significant parts unanswered
< 50 %	Only a small fraction of the question was addressed

SECTION 4

Statistical & Validity Metrics

Objective, formula-based checks that complement the LLM judge

BERTScore F1 %

EVALUATION SHEET

WHAT IT MEASURES

A semantic similarity score measuring how closely the chatbot's answer mirrors the retrieved context at the sentence level. Unlike simple word matching, it understands meaning — 'students passed' and 'learners succeeded' are treated as similar even though no words overlap.

FORMULA

Step 1 – split answer and context into individual sentences

1 n

Precision = $\frac{1}{n} \sum_{i=1}^n \max_j \cos(\text{answer_sent_i}, \text{context_sent_j})$

n i=1 j

1 m

Recall = $\frac{1}{m} \sum_{j=1}^m \max_i \cos(\text{context_sent_j}, \text{answer_sent_i})$

m j=1 i

$2 \times \text{Precision} \times \text{Recall}$

F1 = $\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Precision + Recall

BERTScore F1 % = F1 × 100

Weight in Accuracy: 15 %

HOW IT WORKS

Both the answer and the retrieved context are split into individual sentences. Each sentence is encoded into a 384-dimension vector by all-MiniLM-L6-v2. A similarity matrix is built between all answer sentences and all context sentences. Precision measures how well the answer covers the context. Recall measures how well the context is reflected in the answer. F1 balances both into a single number.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
85 – 100 %	Very strong overlap — answer closely mirrors the docs
70 – 84 %	Good match — some paraphrasing or minor omissions
50 – 69 %	Moderate — answer uses very different language to docs
< 50 %	Answer is semantically distant — possible hallucination

■ For SQL/KB answers where context is empty, this defaults to 100 % — evaluated by the LLM judge instead.

Link Validity

EVALUATION
SHEET

WHAT IT MEASURES

Whether the hyperlinks included in the chatbot's response actually work. Broken links frustrate users and indicate that knowledge-base content needs updating. This metric counts how many links are reachable.

FORMULA

Links found = regex: `r'\[.*?\]\((https?:\/\/.*?)\)'`

Valid links = links returning HTTP status code < 400 within 5 s

Score = `valid_count ÷ total_count`

Display = 'X/Y Valid' e.g. '3/4 Valid'

Weight in Accuracy: 0 % (informational only — links are rare)

HOW IT WORKS

After the bot answer is received, the script scans for markdown-format URLs. For each one, an HTTP HEAD request is sent — this is lightweight and only checks that the page exists, without downloading its content. A response of 200–399 marks the link as Valid. Timeouts, 404s, or 5xx errors mark it as broken. Answers with no links show 'N/A'.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
N/A	No hyperlinks in the answer — normal for SQL/KB answers
All Valid	All links live — references are up to date
Partial	Some links broken — update those pages in the knowledge base
0/N Valid	All links broken — knowledge base may have stale URLs

SECTION 5

Final Accuracy Score

One balanced number summarising overall response quality

Accuracy %

EVALUATION SHEET

WHAT IT MEASURES

A single weighted score summarising the overall quality of one chatbot response. It combines four independent quality signals — each measuring a different dimension — so no single factor can mask problems in another. Easy to compare across runs, prompts, and model versions.

FORMULA

Accuracy = $\sum w_i \cdot x_i$ (weighted mean, all x_i in $[0.0, 1.0]$)

$$w_{\text{Faithfulness}} = 0.30 \times x_{\text{Faithfulness}}$$

$$w_{\text{Relevance}} \cdot \text{Relevance} = 0.25 \times x_{\text{Relevance}}$$

$w_{\text{Completeness}} = 0.20 \times x_{\text{Completeness}}$

$$w_{\text{BERTScore F1}} = 0.15 \times x_{\text{BERTScore F1}}$$

$$w_{\text{Source}} \cdot \text{Source Score} = 0.10 \times x_{\text{Source}}$$

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

$$\sum w_i = 1.00 \text{ (weights always sum to 1)}$$

$$\text{Accuracy \%} = \text{Accuracy} \times 100$$

Source Score: 1.0 = correct pipeline | 0.75 = neutral | 0.0 = unknown

HOW IT WORKS

Faithfulness carries the highest weight (30 %) because a hallucinated answer is worse than an incomplete one. Relevance (25 %) rewards staying on topic. Completeness (20 %) rewards coverage. BERTScore (15 %) adds an objective statistical counterweight to the LLM judge. Source Appropriateness (10 %) gives a small bonus when the right pipeline handled the query — e.g., SQL for 'list all students', RAG for policies.

HOW TO READ THE RESULT

SCORE	INTERPRETATION
90 – 100 %	Excellent — accurate, complete, grounded, on-topic
75 – 89 %	Good quality — suitable for real-user deployment
60 – 74 %	Acceptable — some improvements needed
40 – 59 %	Poor — investigate retrieval failures or prompt design
< 40 %	Problematic — likely hallucinating or mis-routing

- *Weights can be adjusted in prompt test.py to reflect your team's priorities.*

Weight Breakdown

Metric	Weight	Reason
Faithfulness %	30 %	Hallucination is the biggest risk in RAG systems
Relevance %	25 %	Off-topic answers are useless even if accurate
Completeness %	20 %	Partial answers leave students without key info
BERTScore F1 %	15 %	Objective statistical check alongside LLM judge
Source Score	10 %	Minor routing bonus — correct pipeline improves trust
TOTAL	100 %	